

Documentazione_Sistemi_Operativi

Documentazione Scelte Implementative - Esame di Sistemi Operativi

Progetto: Dungeon Explorer

- 1. Architettura di Sistema Generale: Multi-Thread e Multi-Process
- 2. Il Modello del Matchmaker: Multithreading e Mutex
 - 2.1 Creazione Indipendente (Thread Detach) e Allocazione Dinamica
- 3 Dalla Lobby al Game
- 4. Monitoraggio: Il "Game Monitor Loop"
- 5. L'Architettura interna di Game.c: I/O Multiplexing (poll)
 - 1. Inizializzazione della Partita
 - 2. Il Loop della Partita
 - 3. Fase di Fine Partita

La presente documentazione illustra e motiva le scelte implementative applicate nello sviluppo del server per il progetto *Dungeon Explorer*. L'analisi si concentra sulla gestione dei processi, dei thread, della sincronizzazione, dell'I/O di rete, con particolare attenzione ai moduli `matchmaker.c` e `game.c`.

1. Architettura di Sistema Generale: Multi-Thread e Multi-Process

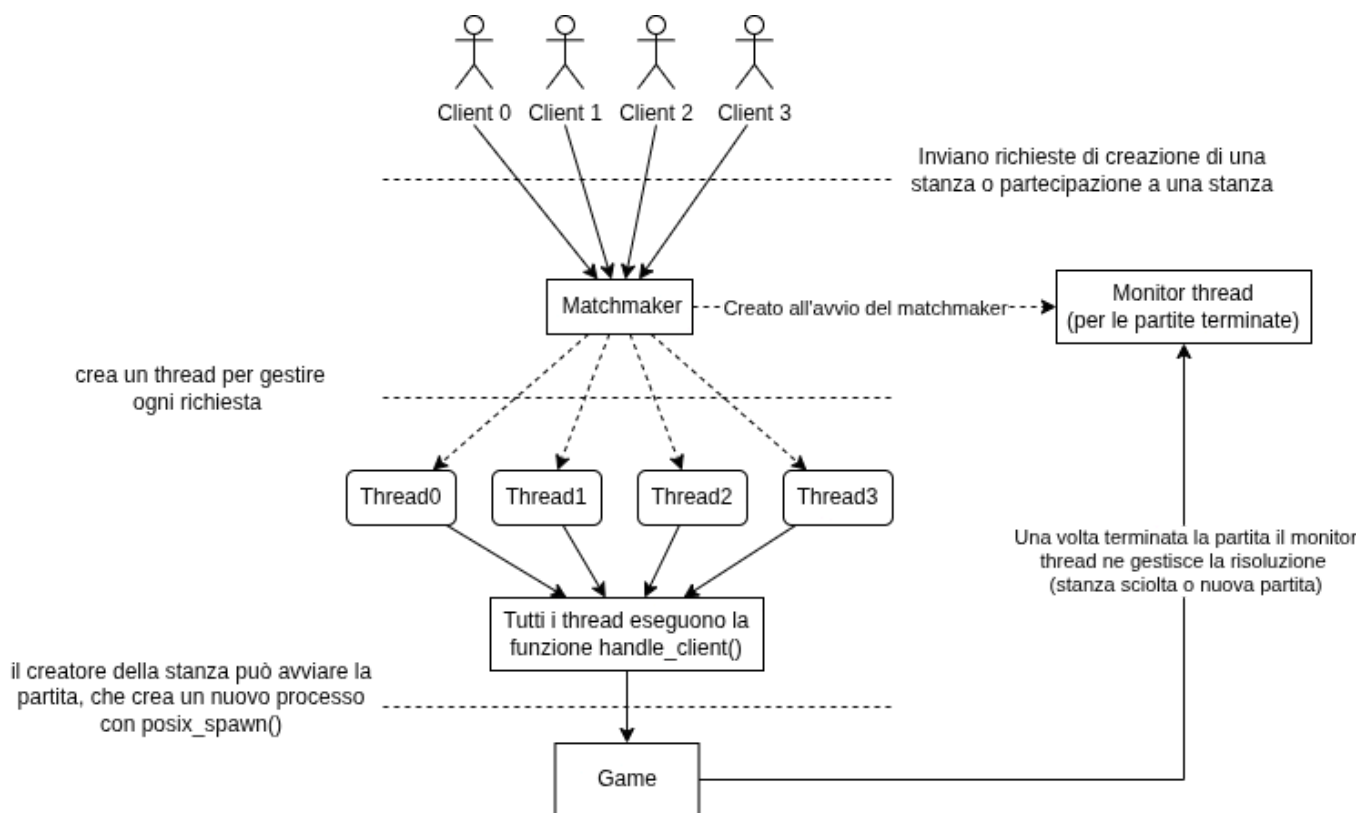
L'infrastruttura backend del gioco è stato suddiviso in due entità separate, aventi ruoli e cicli di vita diametralmente opposti:

1. **IL Matchmaker** (`matchmaker.c`): Rappresenta il demone server primario. È una singola istanza sempre attiva che ascolta connessioni in ingresso, accetta i client e manipola le stanze d'attesa (lobby). È progettato con un'architettura **Single-Threaded-Event-Loop** con l'aggiunta di un **Monitor Thread**.
2. **L'Istanza di Gioco** (`game.c`): È il cuore del gameplay. Viene istanziato per ogni partita che viene avviata. È progettato come un **Processo Separato**, isolato dagli altri, e con una logica a singolo thread basata sull'**I/O Multiplexing**.

Il protocollo di trasporto da noi utilizzato è TCP, questa scelta particolare (dato che la maggior parte dei giochi utilizza UDP) è stata dettata dal fatto che Dungeon Explorer è un **gioco a turni** quindi:

- La velocità di invio dei messaggi non è così importante come in un gioco real-time.
- Essendo la maggior parte del gameplay incentrato su delle scelte, l'arrivo sicuro di pacchetti integri e in ordine è molto più importante.

Qui di seguito riportiamo un grafico rappresentante la struttura che assume il server, e quali thread/processi vengono creati:



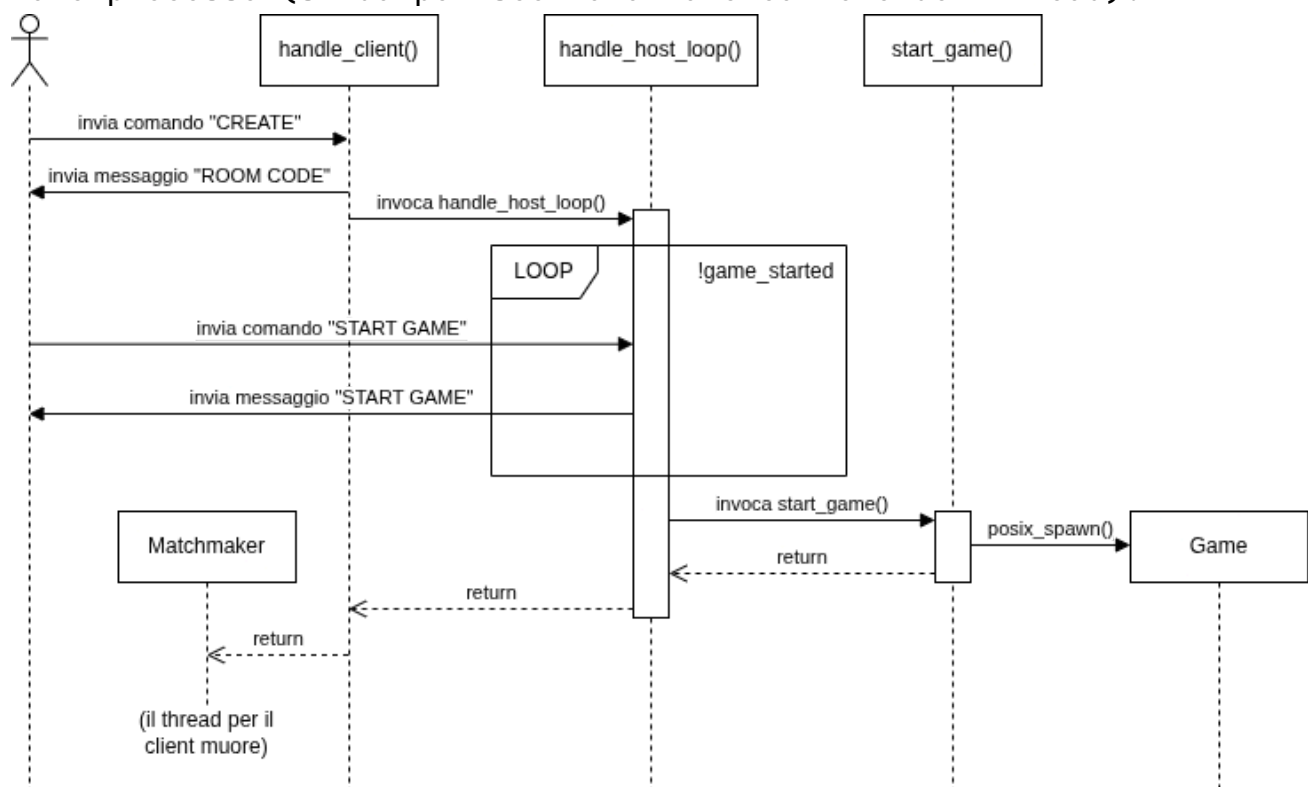
2. Il Modello del Matchmaker: Multithreading e Mutex

Il Matchmaker deve gestire una moltitudine di client che si collegano e scollegano, creando o aggregandosi alle stanze. Lo stato del sistema (il vettore globale `Game games[MAX_GAMES]`) deve essere visibile e modificabile da chiunque.

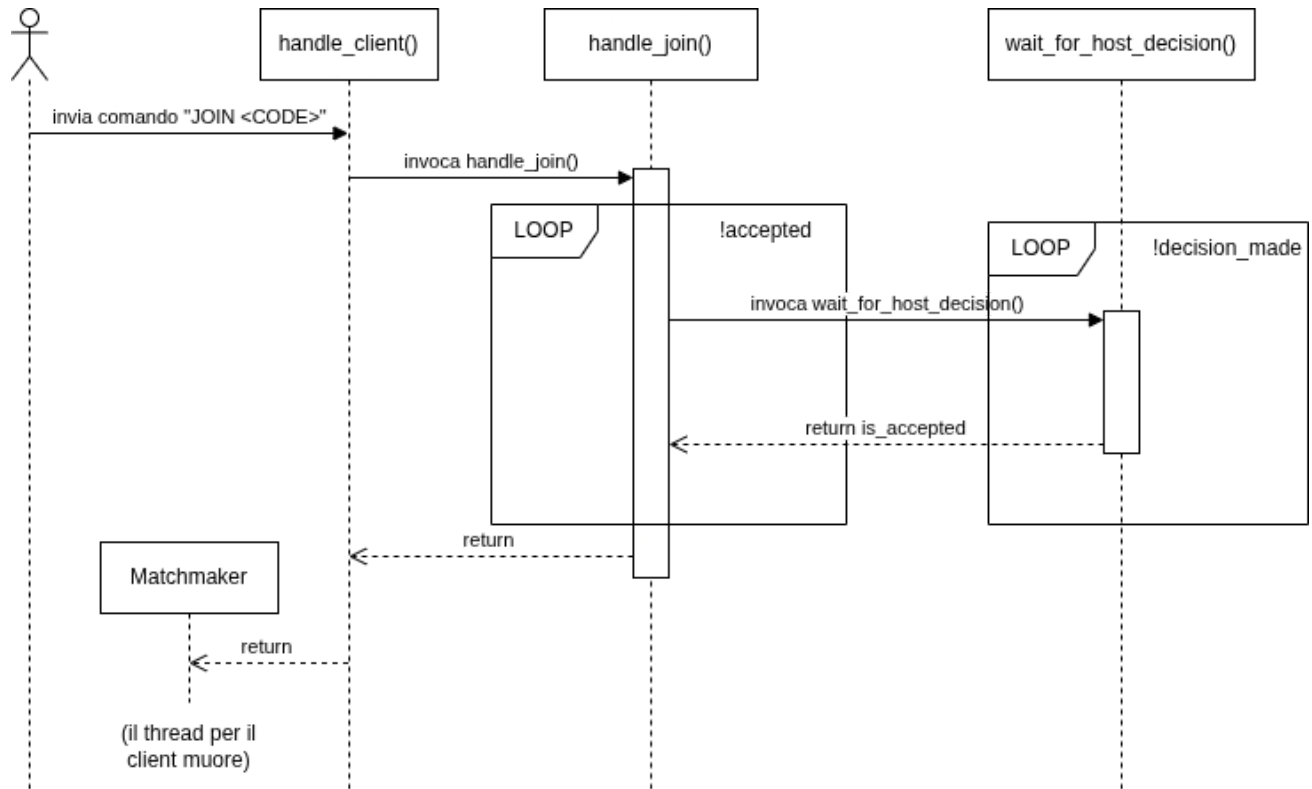
2.1 Creazione Indipendente (Thread Detach) e Allocazione Dinamica

Ogni volta che l'API `accept()` intercetta una nuova connessione sul socket in ascolto, genera un thread con la funzione `pthread_create` alla quale vi è associata la funzione `handle_client()`. La funzione aspetta di ricevere un messaggio dal frontend (mediante la `recv()`) per agire di conseguenza:

- Se il messaggio ricevuto è `CREATE` viene aggiornato l'array globale `games[]` entrando in un mutex, così se altri utenti provano a creare partite non scriveranno nella stessa cella di memoria. Quindi si entra nella funzione `handle_host_loop()` che ha il compito di accettare/rifiutare nuovi client o avviare la partita (minimo 2 giocatori). Una volta avviata la partita il thread muore. Il seguente sequence diagram illustra più in dettaglio come avviene tale processo (si dà per scontata la creazione del thread):



- Se il messaggio è `JOIN <codice-stanza>`, il giocatore, se ha inserito un codice stanza corretto, aggiunge il suo socket file-descriptor all'interno dell'array games, quindi il suo thread muore. Anche qui riportiamo un sequence diagram per mostrare in maggior dettaglio ciò che succede (e anche qui assumiamo che il thread sia stato creato):



3 Dalla Lobby al Game

L'istante critico in cui un host dichiara esplicitamente lo `START_GAME` viene chiamata una funzione apposita (`start_game()`) per avviare e lanciare un nuovo processo per la nuova partita. In particolare:

1. **Inibisce la chiusura delle socket (`FD_CLOEXEC`):**

Normalmente, quando un programma su base Unix avvia un altro programma (tramite `fork/exec` o `spawn`), i descrittori di file (come le socket di rete) vengono chiusi nel nuovo processo. Questa funzione cicla su tutti i giocatori connessi e rimuove il flag `FD_CLOEXEC` dalle loro socket. In questo modo **le connessioni attive vengono ereditate dal nuovo processo**, permettendo al nuovo eseguibile di interfacciarsi direttamente con i client.

2. **Prepara gli argomenti per il nuovo processo:**

Crea due stringhe concatenando le informazioni dei giocatori connessi:

- `sockets_args`: una lista di tutti i File Descriptor (es: `"4 5 6"`).
- `ids_args`: una lista di tutti gli ID associati ai giocatori (es: `"0 1 2 "`).

3. **Lancia l'eseguibile del gioco:**

Utilizza `posix_spawn` (un'alternativa più moderna ed efficiente a `fork` + `exec`) per lanciare il programma game. A questo programma passa i seguenti argomenti da riga di comando (`argv`):

- Il nome del programma (`"game_server"`)
- Il codice della stanza (es: `"A1B2C3"`)
- La stringa con le socket
- La stringa con gli ID dei giocatori

4. **Salva il PID per il monitoraggio:**

Se l'avvio va a buon fine, la funzione salva il pid del nuovo eseguibile appena creato dentro la variabile `game->game_pid`. Questo passaggio è fondamentale affinché la funzione `game_monitor_loop` (che vedremo nella sezione successiva) possa "sorvegliare" questo specifico processo e accorgersi quando la partita viene terminata.

4. Monitoraggio: Il "Game Monitor Loop"

La funzione `game_monitor_loop` è un thread eseguito in background che agisce da supervisore per i processi di gioco (i processi figli) generati dal server matchmaker.

Ecco i compiti principali che svolge:

1. Monitoraggio dei processi (Non bloccante):

Utilizza un ciclo infinito in cui chiama periodicamente `waitpid(-1, &status, WNOHANG)`. Questo le permette di individuare i processi di gioco che sono terminati senza bloccare l'esecuzione del thread. Tra un controllo e l'altro si mette in pausa per 100ms per non consumare CPU.

2. Identificazione della partita:

Quando rileva un processo terminato, ne cerca il PID nel registro delle partite attive.

3. Gestione della chiusura:

Analizza il motivo per cui il processo figlio è terminato:

- **Se l'Host decide di continuare (Exit code 10):** La funzione riporta la stanza allo stato di "Lobby". Controlla la validità della connessione di ogni partecipante tramite `poll()`. Se un giocatore si è disconnesso chiude la sua socket; ai giocatori ancora attivi invia un messaggio di notifica ("Siete tornati in lobby!"). Infine, riavvia il thread incaricato di ascoltare i comandi dell'host (`host_loop_thread`).
- **Se la partita o la stanza è chiusa regolarmente:** Questo avviene con codici di uscita diversi da 10. La funzione svuota la stanza espellendo tutti i giocatori (chiudendo le rispettive socket) e azzerando la struttura della partita in memoria (`memset`), rendendo lo slot disponibile per nuovi giocatori.
- **Casi di crash:** Se la partita si interrompe in modo anomalo (senza un exit code standard di uscita), forza la pulizia della stanza come nel caso precedente, liberando tutte le risorse e disconnettendo i giocatori per evitare file descriptor "appesi".

5. L'Architettura interna di Game.c: I/O Multiplexing (poll)

Il file `game.c` rappresenta il "motore" del gioco per una specifica partita avviata. Si tratta del loop principale (eseguibile a sé stante) in cui i giocatori affrontano il dungeon, stanza dopo stanza.

5.1. Inizializzazione della Partita

- Al lancio del programma (`game_code` e un elenco di ID e socket di rete vengono passati per riga di comando dal processo genitore, che tipicamente è il "matchmaker"), vengono letti gli argomenti e registrati tutti i giocatori nella partita.
- `init_players()`: Ripristina gli HP dei giocatori, li mette tutti "in vita" e assegna gli oggetti base (Es. a mani nude, senza armatura, ecc.).
- `broadcast_all_players_info()`: Invia ai client connessi tutte le informazioni sullo stato iniziale della squadra.
- `generate_dungeon(...)`: Crea proceduralmente la conformazione del dungeon in memoria e unisce i vari tipi di stanze, posizionando la stanza del Boss in quella più lontana (la fine del dungeon).

5.2. Il Loop della Partita

Entrati nel ciclo, il server eseguirà in sequenza queste tre fasi iterando fintanto che i giocatori avanzano:

1. Gestione della Stanza (`run_room`):

- Viene richiamata la funzione della stanza corrente: `room->encounter(...)`.
- Gli *encounter* sono gestiti nel file `encounter.c` e definiscono che succederà in questa stanza passandovi l'array dei giocatori connessi.
- Possono verificarsi incontri specifici:
 - `combat_encounter()` / `boss_encounter()` per far scontrare a turni su griglia i nemici contro i giocatori
 - `treasure_encounter()` per la raccolta degli equipaggiamenti dai forzieri, dove i giocatori confermeranno se prendere o meno quell'item, oppure per la raccolta di oro.
 - `trap_encounter()` per infliggere danni a un giocatore

- `run_room()` intercetta se l'encounter fallisce (morte di tutti, causando *Game Over*) o se il boss viene sconfitto (Vittoria), chiamando `handle_end_game()`.

2. **Votazione Prossima Stanza** (`vote_next_room`):

Il server inizia componendo una stringa che contiene le porte disponibili (es. nord, sud, est) e lo stato delle stanze adiacenti (completata o non completata). Questo messaggio viene inviato in broadcast a tutti i client.

Normalmente, la lettura di dati da un socket di rete tramite `recv()` è un'operazione bloccante: se chiedo di leggere cosa ha deciso il Giocatore 1, il server si ferma finché il Giocatore 1 non risponde. Se il Giocatore 1 si allontana o gli cade la connessione, l'intera partita si bloccherebbe.

Per evitare questo, il gioco usa il **multiplexing dell'I/O** tramite `poll`.

`poll` permette al server di "ascoltare" contemporaneamente i socket di tutti i giocatori e di impostare un timer massimo di attesa. Viene creato un array di strutture `pollfd`, una per ogni giocatore. A ogni struttura viene assegnato il file descriptor. Se un giocatore è già morto o disconnesso, il suo `fd` viene impostato a `-1` in modo che `poll` lo ignori.

Viene avviato un ciclo `while` che durerà un massimo di 20 secondi e quando `poll` restituisce un numero `> 0`, significa che è successo qualcosa su uno o più socket:

- Se si verifica `POLLIN`, il client ha mandato un messaggio. Il server fa una `recv()` (sapendo di essere non bloccante), estrae il voto (es. `SEND_DECISION NORTH`), incrementa il contatore dei voti per quella direzione e segna che quel giocatore ha votato.
- Se si verifica `POLLHUP` (Hang Up) o `POLLERR`, vuol dire che il client ha chiuso forzatamente la connessione (es. ha `killato` il gioco o è caduta la rete). Il server se ne accorge istantaneamente e chiama `mark_player_disconnected()`, ignorandolo per il resto della votazione.

Il ciclo di `poll` si interrompe se si verificano due condizioni:

1. **Tutti i giocatori vivi hanno votato** (`received == expected_votes`).
2. **Il tempo scade** (`elapsed >= timeout_ms`, passano i 20 secondi).

Usciti dal ciclo, il server conta i voti (`votes[4]`). Se c'è una direzione con la maggioranza assoluta, sceglie quella. In caso di pareggio o se nessuno ha votato estrae casualmente una delle porte disponibili

3. Inoltro della scelta e Aggiornamento:

- Assegna al loop il nuovo indice della stanza da eseguire nel ciclo successivo.
- `build_room_message(...)` e `broadcast(...)` avvisano contemporaneamente i client del tipo di stanza in cui sono entrati.

5.3. Fase di Fine Partita

`handle_end_game()`: Arrivati alla vittoria contro il boss, o a una sconfitta del gruppo, il server interroga l'Host della partita. Gli manda la decisione di continuare (*STAY* - ricominciando/tornando alla lobby) o di chiudere tutto (*LEAVE*).